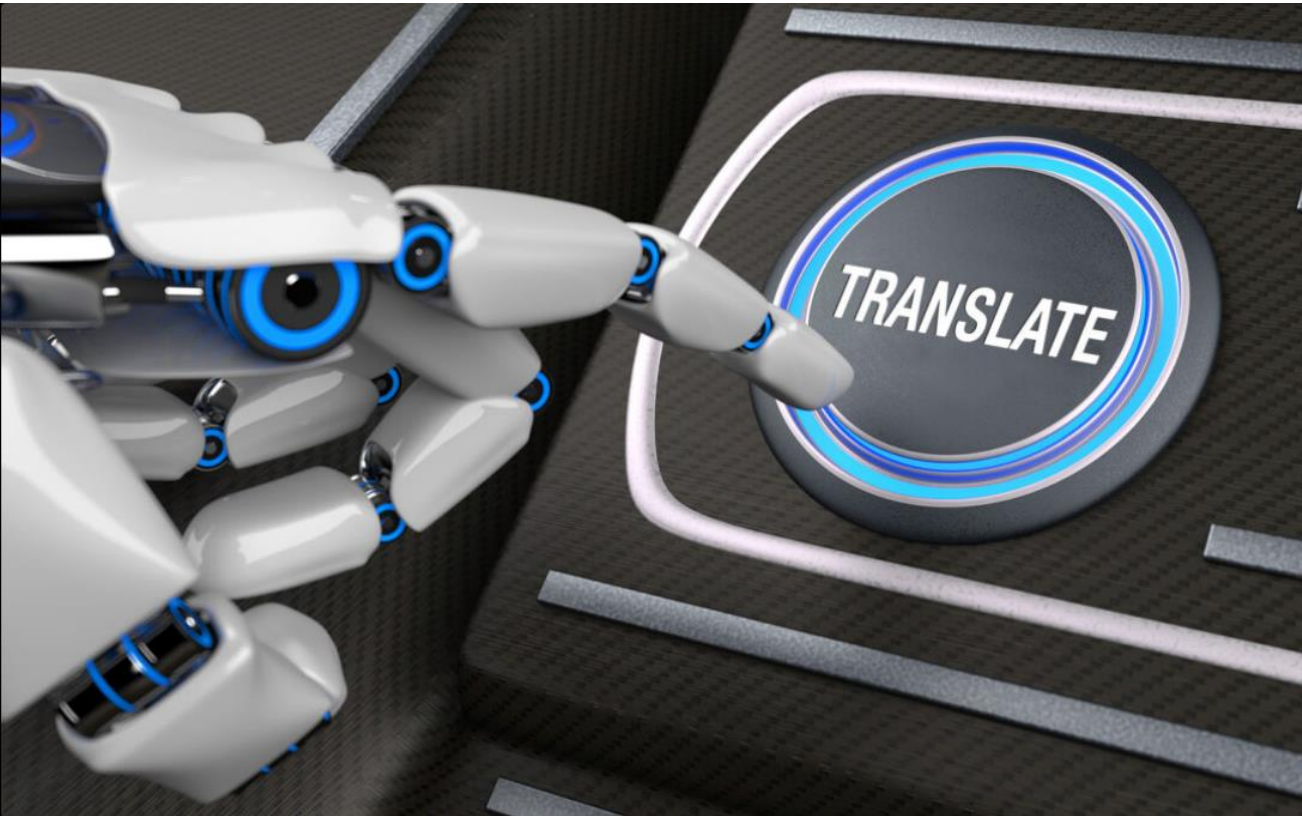


RAPPORT DE PROJET

Application Web de Traduction Intelligente

LinguaSense — Projet AIWave



Module	IA Programming — Campus Ynov
Projet	AIWave / LinguaSense
Type	Application Web de Traduction par IA
Technologies	Node.js, Express.js, Firebase, Gemini API, HTML/CSS/JS
Outil IA de cadrage	Copilot 365
Durée de réalisation	1 semaine (MVP)
Date	Mai 2026

Sommaire

1. **Questions du Client**
2. **Périmètre et Hors Périmètre**
3. **Limites et Contraintes**
4. **Parcours, Workflow et Cible**
5. **Architecture et Technologies**

Explication des choix

6. **Conception de la Base de Données (BD)**
7. **Code et Structure**
8. **Planning et Délais**

1. Introduction

Dans le cadre du module IA Programming à Campus Ynov, nous avons conçu et développé LinguaSense, une application web de traduction intelligente. Ce projet a été réalisé en une semaine et illustre comment l'intelligence artificielle peut être utilisée à la fois comme outil de cadrage produit et comme composant technique central d'une application.

L'idée centrale est de dépasser les traducteurs classiques, trop littéraux, en proposant des traductions plus naturelles et adaptées au contexte. L'application s'adresse aussi bien aux professionnels (marketing, communication, support) qu'au grand public.

L'IA a été mobilisée à deux niveaux distincts : en amont pour structurer le besoin client (via Copilot 365), et en aval comme moteur de traduction (via l'API Gemini de Google).

2. Brief Client & Phase de Cadrage

2.1 Brief initial reçu

"Nous souhaitons créer une application web capable de traduire automatiquement des textes en prenant en compte leur contexte afin d'obtenir des résultats plus naturels et adaptés à l'usage réel. L'objectif est d'aller au-delà d'une simple traduction mot à mot."

Ce brief était volontairement flou et incomplet : aucune contrainte technique définie, aucune liste de langues, aucun périmètre précis. C'est exactement le type de message client que l'on reçoit dans un contexte professionnel réel.

2.2 Utilisation de l'IA pour le cadrage (Copilot 365)

Conformément à la méthode enseignée en cours, nous n'avons pas commencé par coder. L'IA a d'abord servi à transformer ce brief flou en artefacts concrets et exploitables.

Artefact produit par l'IA	Description
Questions de découverte	12 questions classées par thème : objectifs, utilisateurs, fonctionnalités, langues, contraintes...
Brief client amélioré	Reformulation claire du besoin en une phrase produit avec périmètre précis
V1 / Hors-périmètre	Séparation de ce qu'on construit (V1) et ce qu'on reporte (plus tard / jamais)
Parcours utilisateur	Les 12 étapes du parcours : Connexion → Traduction → Historique → Premium
Architecture technique	Stack choisie, rôle de chaque couche, workflow technique en 10 étapes
Critères d'acceptation	Définition de ce qui valide la V1 comme fonctionnelle

2.3 Réponses client & décisions de cadrage

Thème	Réponse / Décision
Objectif	Traductions naturelles et contextualisées (vs traduction mot à mot)
Utilisateurs cibles	Professionnels (marketing, support) + grand public (étudiants, utilisateurs occasionnels)
Langues V1	13 langues : arabe, allemand, anglais, espagnol, français, italien, japonais, coréen, néerlandais, portugais, russe, turc, chinois
Compte utilisateur	Obligatoire (inscription + connexion via Firebase Auth)
Historique	Oui — texte source, traduction, langues, date, suppression possible
Modèle économique	15 traductions gratuites, puis redirection vers upgrade.html (abonnement simple)
Import de fichiers	Non — hors périmètre V1
Paieement réel	Non — hors périmètre V1 (page premium sans paiement intégré)
Assistant IA	Oui — endpoint /api/chat pour poser des questions liées à la traduction

2.4 Périmètre V1 défini

Dans le périmètre :

- Saisie de texte et sélection des langues source et cible
- Champ contexte dans l'interface (amélioration future du prompt)

- Traduction via l'API Gemini avec réponse rapide
- Assistant IA (endpoint /api/chat) pour questions sur la traduction
- Système de comptes utilisateurs (inscription, connexion, déconnexion)
- Historique personnel avec suppression individuelle des entrées
- Contrôle de la limite gratuite (15 traductions) via Firestore
- Page premium upgrade.html présentant les offres
- Interface web simple, moderne, intuitive

Hors périmètre (V1) :

- Import / export de fichiers (PDF, Word)
- Paiement réel en ligne
- Apprentissage automatique basé sur les corrections
- Modèle d'IA personnalisé ou fine-tuning
- API publique, application mobile, intégrations tierces

3. Architecture Technique

3.1 Vue globale

Utilisateur → Frontend (HTML/CSS/JS + Firebase SDK) → Backend (Node.js/Express) → Gemini API + Firebase (Auth & Firestore)

L'architecture suit un modèle 3 tiers classique, avec une séparation nette entre l'interface utilisateur, la logique serveur et les services externes. Le backend Express sert à la fois le frontend statique et l'API REST.

3.2 Stack technologique complète

Couche	Technologie	Rôle
Frontend	HTML, CSS, JavaScript (ES Modules)	Interface utilisateur, appels API, gestion de l'état
Design	Figma	Maquettes de l'interface avant développement
Backend	Node.js + Express.js	Serveur API, routing, middleware, logique métier
IA de traduction	Google Gemini (via @google/generative-ai)	Génération des traductions et réponses assistant
Auth & BDD	Firebase Authentication + Firestore	Comptes utilisateurs, historique, statut abonnement
Auth serveur	Firebase Admin SDK	Vérification des tokens Firebase côté backend
Sécurité	Variables d'environnement (.env)	Protection des clés API (Gemini, Firebase Admin)
Versioning	GitHub	Gestion du code source et collaboration
Cadrage IA	Copilot 365	Questions de découverte, brief amélioré, périmètre

3.3 Structure des fichiers

```
AIWave-projet/
├── backend/
│   ├── controllers/           # translateController.js - logique métier
│   ├── middleware/           # authMiddleware.js - vérification token Firebase
│   ├── routes/               # translateRoutes.js - endpoints API
│   ├── services/             # aiService.js - appels Gemini + fallback
│   ├── server.js             # Point d'entrée Express
│   └── .env                  # Clés API (non versionné)
├── frontend/
│   ├── js/
│   │   ├── firebase.js       # Initialisation Firebase + exports auth/db
│   │   ├── auth.js           # Inscription, connexion, déconnexion, navbar
│   │   ├── translate.js       # Logique de traduction + sauvegarde Firestore
│   │   └── history.js         # Chargement et suppression de l'historique
│   └── css/style.css
```

```

└─ index.html           # Page traducteur principal
└─ login.html           # Connexion
└─ register.html        # Inscription
└─ history.html          # Historique
└─ upgrade.html          # Page premium

```

3.4 Développement du code :

2.FRONTEND :

ASSETS :

1/Logo.png : L'identité visuelle de **StudyShare**. Il assure une interface professionnelle et sert souvent de lien de retour à l'accueil



CSS :

2/Style.css : Ce fichier définit l'**identité visuelle** et la **structure ergonomique** de la plateforme StudyShare. Il repose sur quatre piliers techniques :

Centralisation (:root) : Utilisation de variables CSS pour uniformiser les couleurs (bleu primaire, fonds neutres), les ombres (`--shadow`) et les arrondis (`--radius`). Cela permet une maintenance rapide et une cohérence visuelle sur toutes les pages.

Standardisation (Reset & Body) : Réinitialisation des marges et paddings (*) pour garantir un affichage identique sur tous les navigateurs. Le choix des polices 'Inter' et 'Roboto' assure une lisibilité optimale du contenu académique.

Navigation Persistante (header) : Le bandeau haut est configuré en `position: sticky`, restant accessible lors du défilement. L'utilisation d'un `z-index: 100` évite que le contenu des cours ne passe par-dessus la barre de navigation.

Mise en page Flexible (.navbar) : Emploi de **Flexbox** pour un alignement précis du logo et des liens. Le logo est redimensionné de manière fixe (35px) et bénéficie d'un effet visuel au survol (`opacity: 0.8`) pour améliorer l'expérience utilisateur.

```

frontend > css > # style.css > :root
1  :root {
2    --primary: #2563eb;
3    --primary-dark: #1d4ed8;
4    --bg-body: #f8fafc;
5    --bg-card: #ffffff;
6    --text-main: #1e293b;
7    --text-muted: #64748b;
8    --border-color: #e2e8f0;
9    --shadow: 0 4px 6px -1px rgb(0 0 0 / 0.1), 0 2px 4px -2px rgb(0 0 0 / 0.1);
10   --radius: 12px;
11 }
12
13 * {
14   margin: 0;
15   padding: 0;
16   box-sizing: border-box;
17 }
18
19 body {
20   font-family: 'Inter', 'Roboto', sans-serif;
21   background-color: var(--bg-body);
22   color: var(--text-main);
23   line-height: 1.5;
24 }
25
26 /* Header moderne */
27 header {
28   background-color: var(--bg-card);
29   border-bottom: 1px solid var(--border-color);
30   padding: 0 40px;
31   height: 70px;
32   display: flex;
33   align-items: center;
34   position: sticky;
35   top: 0;
36   z-index: 100;
37 }

```

```

frontend > css > # style.css > :root
38
39 .navbar {
40   width: 100%;
41   max-width: 1200px;
42   margin: 0 auto;
43   display: flex;
44   justify-content: space-between;
45   align-items: center;
46 }
47
48 .logo-container {
49   display: flex;
50   align-items: center;
51   gap: 12px;
52   text-decoration: none;
53   transition: opacity 0.2s;
54 }
55
56 .logo-container:hover {
57   opacity: 0.8;
58 }
59
60 .logo {
61   height: 35px;
62   width: auto;
63 }
64
65 .brand-name {
66   font-weight: 800;
67   font-size: 1.1rem;
68   letter-spacing: -0.5px;
69   color: var(--primary);
70 }
71
72 .nav-links {
73   display: flex;
74   align-items: center;
75   gap: 20px;

```

Js :

3/Auth.js : est le module central de sécurité du frontend. Il sert d'intermédiaire entre l'utilisateur et le serveur pour gérer l'accès à l'application **StudyShare**

Partie 1 :

Importation des outils de sécurité : Elle importe les fonctions essentielles de **Firebase Auth** (pour la connexion/déconnexion) et de **Firestore** (pour la base de données).

Surveillance en temps réel (onAuthStateChanged) : Cette fonction "écoute" en permanence si un utilisateur est connecté ou non. Dès que l'état change (connexion ou déconnexion), le code s'exécute automatiquement.

Mise à jour dynamique de la barre de navigation :

- Si un utilisateur est détecté (`if (user)`), le code transforme dynamiquement le lien "Connexion" en bouton "Déconnexion".
- Il modifie également les liens (`href`) et ajoute des écouteurs d'événements (`click`) pour déclencher la fonction de déconnexion dès que l'utilisateur clique dessus.

```

frontend > js > JS auth.js > ...
1  import { auth } from "../firebase.js";
2  import {
3    createUserWithEmailAndPassword,
4    signInWithEmailAndPassword,
5    signOut,
6    onAuthStateChanged,
7    updateProfile
8  } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
9  import { db } from "../firebase.js";
10 import {
11   collection,
12   setDoc,
13   doc,
14   serverTimestamp
15 } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";
16
17 // — Auth state: update navbar on every page —
18 onAuthStateChanged(auth, (user) => {
19   const navLogin = document.getElementById("navLogin");
20   const navRegister = document.getElementById("navRegister");
21   const navLoginIcon = document.getElementById("navLoginIcon");
22
23   if (user) {
24     // User is logged in – swap login/register links for a Logout button
25     if (navLogin) {
26       navLogin.textContent = "Déconnexion";
27       navLogin.href = "#";
28       navLogin.addEventListener("click", (e) => {
29         e.preventDefault();
30         logout();
31       });
32     }
33     if (navLoginIcon) {
34       navLoginIcon.href = "#";
35       navLoginIcon.addEventListener("click", (e) => {
36         e.preventDefault();
37         logout();

```

Partie 2 :

Capture des données : À la soumission du formulaire (`submit`), il récupère le nom, l'email et le mot de passe saisis par l'étudiant tout en nettoyant les espaces inutiles (`trim()`).

Expérience Utilisateur (UX) : Il désactive le bouton d'inscription et change son texte en "Inscription en cours..." pour éviter les doubles clics et indiquer que le processus est lancé.

Création du compte (Firebase Auth) : Il utilise la fonction `createUserWithEmailAndPassword` pour créer l'identifiant de sécurité. Juste après, il met à jour le profil de l'utilisateur avec son nom (`updateProfile`).

Initialisation dans la base de données (Firestore) : Une fois le compte créé, il génère automatiquement un document dans la collection `users`. Ce document contient :

- Le nom et l'email.
- Un statut par défaut (`isSubscribed: false`).
- La date de création via un timestamp serveur.

Redirection : Si tout se passe bien, l'utilisateur est automatiquement redirigé vers la page d'accueil (`index.html`).

```
18  onAuthStateChanged(auth, (user) => {
39      }
40      if (navRegister) navRegister.style.display = "none";
41  });
42  });
43
44  // --- Register (used in register.html) ---
45  const registerForm = document.getElementById("registerForm");
46  if (registerForm) {
47      registerForm.addEventListener("submit", async (e) => {
48          e.preventDefault();
49
50          const name      = document.getElementById("name").value.trim();
51          const email     = document.getElementById("email").value.trim();
52          const password  = document.getElementById("password").value;
53          const btn       = document.getElementById("registerSubmit");
54
55          btn.disabled = true;
56          btn.textContent = "Inscription en cours...";
57
58          try {
59              const userCredential = await createUserWithEmailAndPassword(auth, email, password);
60              // Save the display name on the Firebase profile
61              await updateProfile(userCredential.user, { displayName: name });
62
63              // Create user document with subscription status (non-subscribed by default)
64              await setDoc(doc(db, "users", userCredential.user.uid), {
65                  displayName: name,
66                  email: email,
67                  isSubscribed: false,
68                  createdAt: serverTimestamp()
69              });
70
71              // Redirect to the translator after successful registration
72              window.location.href = "index.html";
73          } catch (error) {
74              btn.disabled = false;
```

Partie 3 :

Gestion Dynamique (Navbar) : Le code surveille l'état de connexion en temps réel. Si l'étudiant est connecté, il remplace automatiquement les liens "Connexion/Inscription" par un bouton "Déconnexion".

Inscription Sécurisée (Register) : Il crée le compte avec email/mot de passe, met à jour le profil avec le nom de l'étudiant et initialise un document utilisateur dans la base de données Firestore avec un statut par défaut.

Contrôle de Connexion (Login) : Il authentifie l'utilisateur, gère les erreurs de saisie (via `showError`) et protège l'interface en désactivant le bouton pendant le chargement pour éviter les envois multiples.

Sécurité des Sessions (Logout & Redirection) :

- La fonction **Logout** détruit la session active et redirige vers la page de connexion.
- Une **Redirection Intelligente** empêche un utilisateur déjà connecté d'accéder aux pages de login ou register, le renvoyant d'office vers l'accueil.

```
// — Login (used in login.html) —
const loginForm = document.getElementById("loginForm");
if (loginForm) {
  loginForm.addEventListener("submit", async (e) => {
    e.preventDefault();

    const email = document.getElementById("email").value.trim();
    const password = document.getElementById("password").value;
    const btn = document.getElementById("loginSubmit");

    btn.disabled = true;
    btn.textContent = "Connexion en cours...";

    try {
      await signInWithEmailAndPassword(auth, email, password);
      window.location.href = "index.html";
    } catch (error) {
      btn.disabled = false;
      btn.textContent = "Se connecter";
      showError(error.code);
    }
  });
}

// — Logout —
async function logout() {
  await signOut(auth);
  window.location.href = "login.html";
}

// — Redirect logged-in users away from login/register pages —
const isAuthPage = !!document.getElementById("loginPage") || !!document.getElementById("registerPage");
if (isAuthPage) {
  onAuthStateChanged(auth, (user) => {
    if (user) window.location.href = "index.html";
  });
}
```

Partie 4 :

Cette fonction permet de traduire les codes d'erreurs techniques de Firebase en messages clairs et compréhensibles en français.

Dictionnaire de traduction : Le code contient un objet `messages` qui associe chaque code d'erreur Firebase (ex: `auth/wrong-password`) à une phrase explicative (ex: "Mot de passe incorrect.").

Sécurité par défaut : Si un code d'erreur inconnu survient, la fonction affiche un message générique ("Une erreur est survenue...") pour ne pas perturber l'utilisateur.

Gestion dynamique du DOM :

- **Nettoyage** : Avant d'afficher une nouvelle erreur, elle vérifie si un ancien message existe déjà (`authError`) et le supprime pour éviter l'accumulation de texte sur l'écran.
- **Affichage visuel** : Elle crée dynamiquement un nouvel élément paragraphe `<p>`, lui applique un style rouge (`#e53e3e`) pour attirer l'attention, et l'ajoute directement sous le formulaire de connexion ou d'inscription.

```
// — Error messages in French —
function showError(code) {
  const messages = {
    "auth/email-already-in-use": "Cette adresse e-mail est déjà utilisée.",
    "auth/invalid-email": "Adresse e-mail invalide.",
    "auth/weak-password": "Le mot de passe doit contenir au moins 6 caractères.",
    "auth/user-not-found": "Aucun compte trouvé avec cet e-mail.",
    "auth/wrong-password": "Mot de passe incorrect.",
    "auth/invalid-credential": "E-mail ou mot de passe incorrect.",
    "auth/too-many-requests": "Trop de tentatives. Réessayez plus tard.",
  };

  const message = messages[code] || "Une erreur est survenue. Veuillez réessayer.";

  // Remove any existing error message
  const existing = document.getElementById("authError");
  if (existing) existing.remove();

  const errorEl = document.createElement("p");
  errorEl.id = "authError";
  errorEl.style.cssText = "color: #e53e3e; margin-top: 0.75rem; font-size: 0.9rem;";
  errorEl.textContent = message;

  const form = document.getElementById("loginForm") || document.getElementById("registerForm");
  form.appendChild(errorEl);
}
```

4/ `firebase.js` : Ce fichier sert de **passerelle de communication**. Il ne contient pas de logique métier, mais il établit la connexion sécurisée avec ton infrastructure backend.

Importation des SDK : Il charge les modules officiels de Firebase pour l'application globale (`firebase-app.js`), l'authentification (`firebase-auth.js`) et la base de données NoSQL (`firebase-firestore.js`).

Identifiants du projet (`firebaseConfig`) : Il contient les clés techniques nécessaires (API Key, Project ID, App ID) pour que le navigateur sache exactement à quel projet "StudyShare" (ici nommé *linguasense*) il doit envoyer les données.

```
frontend > js > JS firebase.js > ...
1 import { initializeApp } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-app.js";
2 import { getAuth } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
3 import { getFirestore } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";
4
5 const firebaseConfig = {
6   apiKey: "AIzaSyATbYVwCz5lgpdsbbyYNjJlY3LhEiHR30",
7   authDomain: "linguasense-726ba.firebaseio.com",
8   projectId: "linguasense-726ba",
9   storageBucket: "linguasense-726ba.firebaseio.com",
10  messagingSenderId: "371814805702",
11  appId: "1:371814805702:web:92e1ed52c9cb5ed1a027c7",
12  measurementId: "G-Z4511GNKD0"
13 };
14
15 const app = initializeApp(firebaseConfig);
16 export const auth = getAuth(app);
17 export const db = getFirestore(app);
```

5/ History.js : Ce module permet à l'étudiant de consulter et de gérer ses activités passées (traductions ou résumés enregistrés) en communiquant directement avec **Firestore**.

Sécurisation des données (Auth Guard) : Le fichier vérifie systématiquement si un utilisateur est connecté via `onAuthStateChanged`. Si ce n'est pas le cas, il redirige l'individu vers la page de connexion, garantissant la **confidentialité** des données.

Récupération dynamique (loadHistory) : Il interroge la base de données **Firestore** pour récupérer les sous-collections de l'utilisateur (comme les traductions ou résumés). Les résultats sont triés par date décroissante pour afficher les éléments les plus récents en priorité.

Gestion du cycle de vie (deleteTranslation) : Le code permet à l'utilisateur de supprimer définitivement des éléments de son historique. Cette action met à jour simultanément la base de données distante et l'interface visuelle du navigateur sans nécessiter de rechargement de page.

Retour d'information (UX) : Il gère les différents états d'affichage, comme le message de chargement ou l'alerte en cas d'historique vide, afin d'assurer une navigation fluide.

```

frontend > js > JS history.js > ...
1 import { auth, db } from "../firebase.js";
2 import { onAuthStateChanged } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
3 import {
4   collection,
5   getDocs,
6   deleteDoc,
7   doc,
8   query,
9   orderBy
10 } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";
11
12 const historyList = document.getElementById("historyList");
13
14 // — Auth guard + Load history —
15 onAuthStateChanged(auth, async (user) => {
16   if (!user) {
17     window.location.href = "login.html";
18     return;
19   }
20   await loadHistory(user.uid);
21 });
22
23 // — Load and render translations from Firestore —
24 async function loadHistory(uid) {
25   historyList.innerHTML = "<p>Chargement...</p>";
26
27   try {
28     const q = query(
29       collection(db, "users", uid, "translations"),
30       orderBy("timestamp", "desc")
31     );
32
33     const snapshot = await getDocs(q);
34
35     if (snapshot.empty) {
36       historyList.innerHTML = "<p>Aucune traduction sauvegardée pour l'instant.</p>";
37     }
38
39     // Delete button
40     item.querySelector(".btn-delete").addEventListener("click", async () => {
41       await deleteTranslation(uid, docId, item);
42     });
43
44     return item;
45   }
46 }
47
48 // — Delete a translation —
49 async function deleteTranslation(uid, docId, itemEl) {
50   try {
51     await deleteDoc(doc(db, "users", uid, "translations", docId));
52     itemEl.remove();
53
54     // Show empty state if no items left
55     if (historyList.children.length === 0) {
56       historyList.innerHTML = "<p>Aucune traduction sauvegardée pour l'instant.</p>";
57     }
58   } catch (error) {
59     console.error("Erreur lors de la suppression :", error);
60   }
61 }

```

6/translate.js :

Interface de Traduction et IA (translate.js)

Ce module gère les fonctionnalités interactives de StudyShare, notamment la traduction de cours et le chatbot d'assistance.

1. Initialisation et Sécurité (Auth Guard)

Comme on le voit dans la **Capture d'écran 2026-05-14 142638.png**, le fichier commence par lier tous les éléments de l'interface (boutons, zones de texte, sélecteurs de langue) au code JavaScript.

- **Protection** : Il utilise `onAuthStateChanged` pour vérifier que l'utilisateur est bien connecté. Si ce n'est pas le cas, il bloque l'accès et redirige vers la page de connexion.

2. Gestion du Modèle Économique (`initializeCounter`)

La **Capture d'écran 2026-05-14 142651.png** montre une fonctionnalité clé pour la viabilité du projet :

- **Vérification du statut** : Le code récupère le document de l'utilisateur dans Firestore pour vérifier s'il est abonné (`isSubscribed`).
- **Limitation "Free"** : Pour les utilisateurs non-abonnés, il initialise un compteur de traductions. Cela permet de limiter l'utilisation gratuite et d'encourager le passage à une version premium.

3. Communication avec l'IA (API Chat)

La **Capture d'écran 2026-05-14 142708.png** présente la logique d'envoi des messages à l'IA :

- **Sécurisation des requêtes** : Le code génère un jeton d'identification (`getIdToken`) qu'il envoie dans l'en-tête `Authorization` de la requête. Cela prouve au serveur que la demande vient d'un utilisateur authentifié.
- **Appel API** : Il envoie le texte à traduire ou la question au point de terminaison `/api/chat` via une méthode `POST`.
- **Gestion des réponses** : Le script traite la réponse JSON reçue pour l'afficher à l'écran. En cas d'erreur (serveur indisponible ou session expirée), il informe l'utilisateur ou le redirige vers le login.
- **Confidentialité** : L'utilisation de tokens Bearer pour les appels API empêche des tiers d'utiliser l'IA au nom de vos utilisateurs.
- **Intégrité** : Le système de compteur côté Firestore évite qu'un utilisateur ne contourne les limitations de l'application en modifiant simplement son interface locale.
- **Disponibilité** : La gestion rigoureuse des états de chargement (`disabled`, `textContent`) assure que l'application reste stable même lors de ralentissements réseau.

```
frontend > js > JS translatejs > ...
1  import { auth, db } from "../firebase.js";
2  import {
3    |   onAuthStateChanged
4  } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-auth.js";
5  import {
6    |   collection,
7    |   addDoc,
8    |   serverTimestamp,
9    |   getDoc,
10   |   doc,
11   |   getDocs,
12   |   setDoc
13 } from "https://www.gstatic.com/firebasejs/10.12.0/firebase-firestore.js";
14
15 const translateBtn = document.getElementById("translateBtn");
16 const textInput = document.getElementById("text-input");
17 const targetLang = document.getElementById("target-lang");
18 const sourceLang = document.getElementById("source-lang");
19 const contextInput = document.getElementById("context");
20 const outputDiv = document.getElementById("translationOutput");
21 const counterDiv = document.getElementById("translationCounter");
22 const counterValue = document.getElementById("counterValue");
23 const chatForm = document.getElementById("chatForm");
24 const chatInput = document.getElementById("chat-input");
25 const chatMessages = document.getElementById("chatMessages");
26 const chatBtn = document.getElementById("chatBtn");
27
28 console.log("DOM Elements loaded:");
29 console.log("  counterDiv:", counterDiv);
30 console.log("  counterValue:", counterValue);
31 console.log("  translateBtn:", translateBtn);
32
33 // — Auth guard —
34 let currentUser = null;
35
36 onAuthStateChanged(auth, (user) => {
37   if (!user) {
```

Le bloc `try...catch` est une structure de contrôle utilisée pour **gérer les erreurs** sans faire planter l'application. Dans tes fichiers comme `auth.js` ou `translate.js`, il joue un rôle de "filet de sécurité" lors des appels réseau vers Firebase ou ton API

```

try {
  const token = await currentUser.getIdToken();

  const response = await fetch("/api/chat", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${token}`
    },
    body: JSON.stringify({ message })
  });

  if (response.status === 401) {
    window.location.href = "login.html";
    return;
  }

  if (!response.ok) throw new Error(`Erreur serveur : ${response.status}`);

  const data = await response.json();
  loadingMessage.textContent = data.answer || "Aucune reponse recue.";
} catch (error) {
  console.error("Erreur de chat :", error);
  loadingMessage.textContent = `Erreur : ${error.message}`;
  loadingMessage.classList.add("chat-message-error");
} finally {
  chatBtn.disabled = false;
  chatBtn.textContent = "Envoyer";
  chatInput.focus();
}
});

```

7/ **history.html** : Ce fichier définit l'ossature de la page où les étudiants consultent leurs activités passées. Il combine optimisation pour le référencement (SEO) et accessibilité.

1. En-tête et Métadonnées (SEO & Social)

Comme on le voit dans la **Capture d'écran 2026-05-14 143520.png**, le `<head>` est extrêmement complet :

- **Référencement** : Utilisation de balises `meta` pour la description et les mots-clés, ainsi que des liens `canonical` et `alternate` pour gérer le multilingue (français/anglais).
- **Réseaux Sociaux** : Intégration des protocoles **Open Graph** et **Twitter Card** pour assurer un affichage professionnel lors du partage du lien.
- **Performance** : Utilisation de `preconnect` pour charger rapidement les polices Google Fonts.

2. Navigation et Accessibilité

La structure du corps (`<body>`) privilégie la clarté :

- **Barre de Navigation** : Le `<header>` contient un logo cliquable et une barre de navigation (`<nav>`) avec des attributs `aria-label` pour faciliter la lecture par les outils d'accessibilité.
- **Indicateurs visuels** : On y retrouve des liens vers l'accueil, la version premium et un conteneur pour l'icône de profil utilisateur.

3. Zone de Contenu Dynamique

La **Capture d'écran 2026-05-14 143532.png** montre comment la page s'organise pour recevoir les données :

- **Section Historique** : Une balise `<section id="historyList">` sert de conteneur vide. C'est ici que le fichier `history.js` injectera dynamiquement les traductions sauvegardées.
- **État de chargement** : Un paragraphe "Chargement..." est présent par défaut pour informer l'utilisateur pendant que les données sont récupérées depuis Firestore.

4. Connexion des Scripts

En bas de page, le fichier appelle les modules de logique :

- `js/auth.js` : Pour vérifier que l'utilisateur est connecté avant d'afficher l'historique.
- `js/history.js` : Pour récupérer et afficher les documents de la base de données.

```

frontend > <> history.html > ...
1 <!DOCTYPE html>
2 <html lang="fr">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>LinguaSense | Traducteur IA Multilingue</title>
8   <meta name="description"
9     content="LinguaSense est un traducteur IA rapide et intelligent permettant de traduire facilement vos
10   <meta name="keywords"
11     content="traducteur IA, traduction intelligente, traduction multilingue, traduction français anglais">
12   <meta name="author" content="LinguaSense">
13   <meta name="robots" content="index, follow">
14   <link rel="canonical" href="https://www.linguasense.com/">
15   <link rel="alternate" hreflang="fr" href="https://www.linguasense.com/">
16   <link rel="alternate" hreflang="en" href="https://www.linguasense.com/en/">
17   <meta property="og:title" content="LinguaSense | Traducteur IA">
18   <meta property="og:description" content="Traducteur IA multilingue rapide et intelligent.">
19   <meta property="og:type" content="website">
20   <meta property="og:url" content="https://www.linguasense.com/">
21   <meta property="og:image" content="https://www.linguasense.com/assets/Logo.png">
22   <meta name="twitter:card" content="summary_large_image">
23   <link rel="preconnect" href="https://fonts.googleapis.com">
24   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
25   <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet">
26   <link rel="stylesheet" href="css/style.css">
27   <link rel="icon" type="image/png" href="assets/favicon.png">
28 </head>
29
30 <body>
31   <header>
32     <div class="navbar">
33       <a href="index.html" class="logo-container" aria-label="Accueil LinguaSense">
34         
35         <span class="brand-name">LINGUA-SENSE</span>
36       </a>
37       <nav id="mainNav" class="nav-links" aria-label="Navigation principale">
frontend > <> history.html > ...
2 <html lang="fr">
30 <body>
31   <header>
32     <div class="navbar">
33       <nav id="mainNav" class="nav-links" aria-label="Navigation principale">
34         <a id="navHome" href="index.html" class="btn-nav" title="Accueil">Accueil</a>
35         <a id="navUpgrade" href="upgrade.html" class="btn-nav" title="Version premium">Améliorer</a>
36         <div class="user-icon">
37           <a id="navLogin" href="login.html" title="Se connecter">
38             
40           </a>
41         </div>
42       </nav>
43     </div>
44   </header>
45
46   <main id="historyPage" class="history-container">
47     <h1 id="historyTitle">Historique</h1>
48
49     <!-- Items are injected dynamically by history.js -->
50     <section id="historyList" class="history-list">
51       <p>Chargement...</p>
52     </section>
53   </main>
54
55   <footer class="footer">
56     <p>&copy; 2026 LinguaSense. Tous droits réservés.</p>
57   </footer>
58
59   <!-- Firebase; must be type="module" -->
60   <script type="module" src="js/auth.js"></script>
61   <script type="module" src="js/history.js"></script>
62 </body>
63 </html>

```

8/index.html : Ce fichier sert de point d'entrée unique qui définit l'identité visuelle et technique de la plateforme. Comme il s'agit d'une **Single Page Application (SPA)**, il charge les fondations sur lesquelles le contenu dynamique viendra s'injecter.

Détails de la Configuration (Le <head>)

La capture d'écran montre une configuration extrêmement soignée de l'en-tête, axée sur trois piliers :

- **Référencement & SEO :**
 - Les balises `meta` (description, keywords) et le titre définissent comment le site apparaît sur Google.
 - Les liens `canonical` et `alternate` gèrent la structure multilingue (français/anglais), évitant ainsi le contenu dupliqué.
- **Visibilité Sociale (Open Graph & Twitter Cards) :**
 - Les balises `og:title`, `og:image`, et `og:description` assurent que lorsque l'étudiant partage le lien de StudyShare sur les réseaux sociaux, une belle carte avec le logo et une description professionnelle s'affiche automatiquement.
- **Performance & Design :**
 - **Preconnect :** Les lignes 23 et 24 demandent au navigateur de se connecter en avance aux serveurs de polices Google, ce qui accélère le chargement visuel de la page.
 - **Feuilles de Style :** Il lie le fichier `css/style.css` qui définit l'apparence globale (couleurs, polices Inter, mise en page).



```

1 <!DOCTYPE html>
2 <html lang="fr">
3
4 <head>
5   <meta charset="UTF-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>LinguaSense | Traducteur IA Multilingue</title>
8   <meta name="description"
9     content="LinguaSense est un traducteur IA rapide et intelligent permettant de traduire facilement"
10  </meta>
11   <meta name="keywords"
12     content="traducteur IA, traduction intelligente, traduction multilingue, traduction français à"
13  </meta>
14   <meta name="author" content="LinguaSense">
15   <meta name="robots" content="index, follow">
16   <link rel="canonical" href="https://www.linguasense.com/">
17   <link rel="alternate" hreflang="fr" href="https://www.linguasense.com/">
18   <link rel="alternate" hreflang="en" href="https://www.linguasense.com/en/">
19   <meta property="og:title" content="LinguaSense | Traducteur IA">
20   <meta property="og:description" content="Traducteur IA multilingue rapide et intelligent.">
21   <meta property="og:type" content="website">
22   <meta property="og:url" content="https://www.linguasense.com/">
23   <meta property="og:image" content="https://www.linguasense.com/assets/Logo.png">
24   <meta name="twitter:card" content="summary_large_image">
25   <link rel="preconnect" href="https://fonts.googleapis.com">
26   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
27   <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet">
28   <link rel="stylesheet" href="css/style.css">
29   <link rel="icon" type="image/png" href="assets/favicon.png">
30 </head>
  
```

9/ **login.html** : Le fichier `login.html` constitue l'interface d'authentification de **StudyShare**. Son rôle est de fournir un environnement sécurisé et optimisé permettant aux utilisateurs de s'identifier pour accéder à leurs services personnalisés.

1. Optimisation et Identité (Le `<head>`)

Comme on peut le voir dans la **image_930977.png**, ce fichier partage une structure de métadonnées cohérente avec le reste de l'application :

- **SEO et Langues :** Il utilise des balises `meta` pour la description et les mots-clés, ainsi que des attributs `hreflang` pour supporter les versions française et anglaise de la plateforme.
- **Partage Social :** Les protocoles **Open Graph** et **Twitter Card** sont configurés pour maintenir une image de marque professionnelle lors du partage de l'URL de connexion.

- **Performance Visuelle** : L'utilisation de `preconnect` (lignes 23-24) assure que les polices de caractères se chargent sans délai, garantissant que l'utilisateur voit immédiatement un formulaire net et lisible.

2. Structure de Navigation Cohérente

La partie visible du `<body>` montre que l'utilisateur conserve ses points de repère :

- **Barre de Navigation** : On retrouve le logo de l'application et la structure de navigation principale (`mainNav`). Cela permet à un utilisateur qui s'est trompé de page de retourner facilement à l'accueil via le lien `index.html`.
- **Accessibilité** : L'utilisation d'attributs tels que `aria-label` sur le conteneur du logo et la navigation confirme que la page est conçue pour être accessible aux étudiants utilisant des lecteurs d'écran.

```

frontend > < login.html > ...
1  <!DOCTYPE html>
2  <html lang="fr">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>LinguaSense | Connexion</title>
8      <meta name="description"
9          content="LinguaSense est un traducteur IA rapide et intelligent permettant de traduire facilement son
10      <meta name="keywords"
11          content="traducteur IA, traduction intelligente, traduction multilingue, traduction français-anglais">
12      <meta name="author" content="LinguaSense">
13      <meta name="robots" content="index, follow">
14      <link rel="canonical" href="https://www.linguasense.com/">
15      <link rel="alternate" hreflang="fr" href="https://www.linguasense.com/">
16      <link rel="alternate" hreflang="en" href="https://www.linguasense.com/en/">
17      <meta property="og:title" content="LinguaSense | Traducteur IA">
18      <meta property="og:description" content="Traducteur IA multilingue rapide et intelligent.">
19      <meta property="og:type" content="website">
20      <meta property="og:url" content="https://www.linguasense.com/">
21      <meta property="og:image" content="https://www.linguasense.com/assets/Logo.png">
22      <meta name="twitter:card" content="summary_large_image">
23      <link rel="preconnect" href="https://fonts.googleapis.com">
24      <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
25      <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet">
26      <link rel="stylesheet" href="css/style.css">
27      <link rel="icon" type="image/png" href="assets/favicon.png">
28  </head>
29
30  <body>
31      <header>
32          <div class="navbar">
33              <a href="index.html" class="logo-container" aria-label="Accueil LinguaSense">
34                  
35                  <span class="brand-name">LINGUASENSE</span>
36              </a>
37              <nav id="mainNav" class="nav-links" aria-label="Navigation principale">

```

10/ `register.html` : Le fichier `register.html` est la page dédiée à l'enrôlement des nouveaux utilisateurs sur la plateforme StudyShare (LinguaSense). Elle est conçue pour collecter les informations nécessaires à la création d'un profil sécurisé.

1. Configuration Technique et SEO

Comme on le voit dans la **Capture d'écran 2026-05-14 145214.png**, l'en-tête (`<head>`) est optimisé pour la performance et la visibilité :

- **Identité et Référencement** : Le titre "Inscription" et les balises `meta` assurent un bon classement dans les moteurs de recherche.
- **Multilingue** : Les balises `alternate` et `hreflang` permettent de basculer facilement entre les versions française et anglaise de la page.

- **Optimisation visuelle** : Le chargement des polices via `preconnect` et l'appel au fichier `css/style.css` garantissent une interface soignée dès l'ouverture.

2. Structure du Formulaire d'Inscription

La Capture d'écran 2026-05-14 145228.png montre le cœur de la page, situé dans la balise `<main>` :

- **Collecte de données** : Le formulaire (`registerForm`) contient trois champs obligatoires (`required`) : le nom complet, l'adresse e-mail et le mot de passe.
- **Expérience Utilisateur (UX)** : Des étiquettes (`label`) et des textes d'exemple (`placeholder`) guident l'étudiant pour remplir correctement les informations.
- **Interactivité** : Un bouton "S'inscrire" déclenche la logique de création de compte, tandis qu'un lien de redirection permet aux utilisateurs possédant déjà un compte de se rendre rapidement sur la page `login.html`.

3. Logique de Sécurité (Backend Connection)

En bas de fichier, l'appel au script suivant est crucial :

- `js/auth.js` : Ce module JavaScript récupère les données saisies dans le formulaire pour les envoyer à Firebase, gérant ainsi la création effective de l'identifiant de sécurité.

```
frontend > register.html > ...
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>LinguaSense | Inscription</title>
7   <meta name="description"
8     content="LinguaSense est un traducteur IA rapide et intelligent permettant de traduire facilement"
9   <meta name="keywords"
10    content="traducteur IA, traduction intelligente, traduction multilingue, traduction français anglais"
11   <meta name="author" content="LinguaSense">
12   <meta name="robots" content="index, follow">
13   <link rel="canonical" href="https://www.linguasense.com/">
14   <link rel="alternate" hreflang="fr" href="https://www.linguasense.com/">
15   <link rel="alternate" hreflang="en" href="https://www.linguasense.com/en/">
16   <meta property="og:title" content="LinguaSense | Traducteur IA">
17   <meta property="og:description" content="Traducteur IA multilingue rapide et intelligent.">
18   <meta property="og:type" content="website">
19   <meta property="og:url" content="https://www.linguasense.com/">
20   <meta property="og:image" content="https://www.linguasense.com/assets/Logo.png">
21   <meta name="twitter:card" content="summary_large_image">
22   <link rel="preconnect" href="https://fonts.googleapis.com">
23   <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
24   <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet">
25   <link rel="stylesheet" href="css/style.css">
26   <link rel="icon" type="image/png" href="assets/favicon.png">
27 </head>
28 <body>
29   <header>
30     <div class="navbar">
31       <a href="index.html" class="logo-container" aria-label="Accueil LinguaSense">
32         
33         <span class="brand-name">LINGUA-SENSE</span>
34       </a>
35       <nav id="mainNav" class="nav-links" aria-label="Navigation principale">
36         <a id="navLogin" href="login.html" class="btn-nav" title="Se connecter">Se connecter</a>
37         <a id="navUpgrade" href="upgrade.html" class="btn-nav" title="Version premium">Améliorer
```

Ce code définit le squelette visuel pour l'inscription des nouveaux utilisateurs.

- **En-tête (<header>)** : Il contient la barre de navigation principale avec un logo et des liens d'accès rapides pour l'utilisateur.
- **Contenu Principal (<main>)** : C'est le cœur de la page, organisé dans un conteneur nommé `login-container`.
 - **Formulaire d'inscription (<form>)** : Il possède l'identifiant `registerForm` et regroupe les champs de saisie nécessaires : nom complet, adresse e-mail et mot de passe.
 - **Validation et Sécurité** : Chaque champ utilise l'attribut `required` pour forcer la saisie avant l'envoi, et le champ mot de passe est de type `password` pour masquer les caractères saisis.
 - **Bouton d'action** : Un bouton de type `submit` permet d'envoyer les informations.
 - **Lien de redirection** : Un paragraphe propose aux utilisateurs possédant déjà un compte de se rediriger vers la page `login.html`.
- **Pied de page (<footer>)** : Affiche les mentions de copyright pour l'année 2026.
- **Logique de Script** : Le fichier appelle en dernier le script `js/auth.js`, qui est responsable du traitement des données saisies et de la communication avec Firebase.

```

frontend > <> register.html > ...
 2  <html lang="fr">
28  <body>
29    <header>
30      <div class="navbar">
31        <nav id="mainNav" class="nav-links" aria-label="Navigation principale">
32          <a id="navLinguaSense" href="login.html" title="Se connecter">
33            
35          </a>
36        </nav>
37      </div>
38    </header>
39
40    <main id="registerPage">
41      <div class="login-container">
42        <h1 id="registerTitle">Inscription à LinguaSense</h1>
43        <form id="registerForm" class="login-form">
44          <label for="name">Nom complet</label>
45          <input type="text" id="name" name="name" required placeholder="Entrez votre nom complet">
46          <label for="email">Adresse e-mail</label>
47          <input type="email" id="email" name="email" required placeholder="Entrez votre adresse e-mail">
48          <label for="password">Mot de passe</label>
49          <input type="password" id="password" name="password" required placeholder="Entrez votre mot de passe">
50          <button id="registerSubmit" type="submit" class="btn-login">S'inscrire</button>
51        </form>
52        <p id="loginLink" class="login-link">Déjà un compte ? <a href="login.html">Se connecter</a>
53      </div>
54    </main>
55
56    <footer class="footer">
57      <p>&copy; 2026 LinguaSense. Tous droits réservés.</p>
58    </footer>
59
60    <!-- Firebase: must be type="module" -->
61    <script type="module" src="js/auth.js"></script>
62  </body>

```

11 / `upgrade.html` : Le fichier `upgrade.html` constitue l'interface commerciale de la plateforme. Son rôle est de présenter les avantages de la version premium et de permettre aux utilisateurs de faire évoluer leur abonnement pour lever les limitations de traduction.

```

frontend > > upgrade.html > html > body > main#upgradeMain > div#upgradePage.upgrade-page > div#plansGrid.plans-grid > article
1  <!DOCTYPE html>
2  <html lang="fr">
3    <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>LinguaSense | Améliorez votre expérience de traduction</title>
7      <meta name="description"
8        content="Découvrez les avantages de la version premium de LinguaSense pour une expérience
9      <meta name="keywords"
10     content="LinguaSense premium, avantages version payante, traduction illimitée, fonctionnalités avancées"
11     <meta name="author" content="LinguaSense">
12     <meta name="robots" content="index, follow">
13     <link rel="canonical" href="https://www.linguasense.com/upgrade.html">
14     <link rel="alternate" hreflang="fr" href="https://www.linguasense.com/upgrade.html">
15     <link rel="alternate" hreflang="en" href="https://www.linguasense.com/en/upgrade.html">
16     <meta property="og:title" content="LinguaSense | Améliorez votre expérience de traduction">
17     <meta property="og:description" content="Découvrez les avantages de la version premium de LinguaSense"
18     <meta property="og:type" content="website" Follow link (ctrl + click)
19     <meta property="og:url" content="https://www.linguasense.com/upgrade.html">
20     <meta property="og:image" content="https://www.linguasense.com/assets/Logo.png">
21     <meta name="twitter:card" content="summary_large_image">
22     <link rel="preconnect" href="https://fonts.googleapis.com">
23     <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
24     <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800&display=swap" rel="stylesheet">
25     <link rel="stylesheet" href="css/style.css">
26     <link rel="icon" type="image/png" href="assets/favicon.png">
27   </head>
28   <body>
29     <header>
30       <div class="navbar">
31         <a href="index.html" class="logo-container" aria-label="Accueil LinguaSense">
32           
33           <span class="brand-name">LINGUA-SENSE</span>
34         </a>
35         <nav id="mainNav" class="nav-links" aria-label="Navigation principale">
36           <a id="navHistory" href="history.html" class="btn-nav" title="Historique des traductions">
37           <a id="navUpgrade" href="upgrade.html" class="btn-nav active" title="Version premium | Améliorez votre expérience de traduction">

```

Architecture Backend & Sécurité :

Après avoir exploré l'interface utilisateur, nous allons maintenant aborder le "cerveau" de **StudyShare**. Pour garantir une plateforme performante et sécurisée, j'ai fait le choix d'une architecture **Serverless** basée sur **Firebase**. Cette approche permet de déléguer la gestion complexe de l'infrastructure à Google tout en gardant un contrôle total sur la logique métier et la sécurité des données.

3.Backend :

1/ translatecontroller.js : Le rôle de ce fichier est de piloter le processus de traduction. Il agit comme un intermédiaire entre la requête de l'étudiant et le service d'Intelligence Artificielle.

1. Validation et Sécurité des Données

Dès la réception d'une requête, le code effectue deux vérifications essentielles comme on le voit dans la **Capture d'écran 2026-05-14 145952.png** :

- **Validation des entrées** (Lignes 11-16) : Il vérifie que le texte et la langue cible sont bien présents. Sinon, il renvoie une erreur 400.
- **Identification** (Ligne 8) : Il récupère l'identifiant unique de l'utilisateur (`userId`) via le jeton d'authentification pour garantir que la requête est légitime.

2. Gestion du Quota et de l'Abonnement (Business Logic)

C'est ici que l'application applique ses règles commerciales (Lignes 18-36) :

- **Vérification du statut Premium** : Le code interroge **Firestore** pour savoir si l'utilisateur est abonné (`isSubscribed`).
- **Calcul du Quota Gratuit** : Si l'utilisateur n'est pas abonné, le contrôleur compte le nombre de traductions déjà effectuées dans sa collection personnelle.
- **Limitation** : Il définit une limite de **15 traductions** pour les utilisateurs gratuits, calculant dynamiquement le nombre de traductions restantes.

3. Exécution et Réponse

Dans la **Capture d'écran 2026-05-14 150013.png**, une fois les vérifications passées :

- **Appel à l'IA** (Ligne 50) : Le contrôleur délègue la traduction au service `aiService`.
- **Retour d'Information** : Il renvoie au frontend un objet JSON complet contenant le texte traduit, mais aussi les statistiques mises à jour (compteur de traductions et solde restant).
- **Gestion des Erreurs** (Lignes 63-68) : Un bloc `try...catch` intercepte tout problème (panne de l'IA, erreur réseau) et renvoie un message d'erreur clair sans faire planter le serveur.


```

backend > controllers > JS translateController.js > ...
1  const aiService = require('../services/aiService');
2  const admin = require('firebase-admin');
3
4  class TranslateController {
5      async translate(req, res) {
6          try {
7              const { text, language, context } = req.body;
8              const userId = req.user.uid;
9
10             // Validation des données d'entrée
11             if (!text || !language) {
12                 return res.status(400).json({
13                     error: 'Paramètres manquants',
14                     message: 'Les champs "text" et "language" sont requis'
15                 });
16             }
17
18             // Check user subscription status and translation count
19             const db = admin.firestore();
20             const userDoc = await db.collection('users').doc(userId).get();
21             const isSubscribed = userDoc.exists && userDoc.data().isSubscribed === true;
22
23             let translationCount = 0;
24             let remainingTranslations = 15;
25
26             if (isSubscribed) {
27                 const translationsSnapshot = await db
28                     .collection('users')
29                     .doc(userId)
30                     .collection('translations')
31                     .count()
32                     .get();
33
34                 translationCount = translationsSnapshot.data().count;
35                 remainingTranslations = Math.max(0, 15 - translationCount);
36
37                 // Limit non-subscribed users to 15 translations
38             }
39
40             class TranslateController {
41             5      async translate(req, res) {
42
43             48
44             49             // Appel du service AI
45             50             const translatedText = await aiService.translateText(text, language, context);
46
47             51
48             52             res.json({
49             53                 success: true,
50             54                 originalText: text,
51             55                 translatedText: translatedText,
52             56                 language: language,
53             57                 context: context || '',
54             58                 isSubscribed: isSubscribed,
55             59                 translationCount: translationCount + 1,
56             60                 remainingTranslations: remainingTranslations - 1
57             61             });
58
59             62
60             63             } catch (error) {
61             64                 console.error('Erreur dans translateController:', error);
62             65                 res.status(500).json({
63             66                     error: 'Erreur de traduction',
64             67                     message: error.message
65             68                 });
66             69             }
67             70         }
68
69             71
70             72             async chat(req, res) {
71             73                 try {
72             74                     const { message } = req.body;
73             75
74             76                     // Validation des données d'entrée
75             77                     if (!message) {
76             78                         return res.status(400).json({
77             79                             error: 'Paramètre manquant',
78             80                             message: 'Le champ "message" est requis'
79             81                         });
80             82                     }

```

2/ **AuthMiddleware.js** : Ce fichier, **AuthMiddleware.js**, présenté dans la **image_928916.png**, est l'un des composants les plus critiques pour la sécurité de ton backend. Il agit comme un "videur" qui vérifie l'identité de chaque utilisateur avant de le laisser accéder aux ressources sensibles.

Voici un résumé de son fonctionnement que tu peux utiliser pour ta présentation :

Le Gardien des Accès : **AuthMiddleware.js**

Ce script implémente une vérification systématique des jetons d'authentification (**Bearer Tokens**) pour protéger les routes de l'API.

1. Initialisation Sécurisée (Admin SDK)

Comme illustré dans la **image_928916.png**, le fichier commence par configurer l'accès privilégié à Firebase :

- **Protection des secrets** (Lignes 5-11) : Il utilise des variables d'environnement (`process.env`) pour charger les clés privées. C'est une excellente pratique de sécurité car cela évite d'exposer les identifiants réels dans le code source.
- **Initialisation unique** : Le code vérifie si une instance de l'application Firebase existe déjà pour éviter des erreurs de duplication au démarrage du serveur.

2. La fonction de vérification `verifyToken`

Cette fonction est le middleware qui intercepte chaque requête entrante :

- **Extraction du Jeton** (Lignes 16-26) : Elle cherche le jeton dans l'en-tête `Authorization`. Si l'en-tête est manquant ou ne commence pas par "Bearer ", l'accès est immédiatement refusé avec une erreur **401 (Non autorisé)**.
- **Validation par Firebase** (Ligne 29) : Le jeton est envoyé à Firebase Admin pour vérification. Cela garantit que le jeton est authentique, qu'il appartient bien à ton projet et qu'il n'a pas expiré.
- **Injection de contexte** (Ligne 31) : Une fois validé, les informations de l'utilisateur (comme son UID) sont attachées à l'objet de la requête (`req.user`). Cela permet aux contrôleurs (comme le `translateController`) de savoir exactement quel étudiant fait la demande.

3. Gestion Robuste des Erreurs

Le bloc `catch` (Lignes 33-37) assure qu'aucun utilisateur avec un jeton invalide ou expiré ne puisse passer. Il renvoie un message clair "Token invalide ou expiré", protégeant ainsi l'application contre les tentatives d'usurpation d'identité.

```

backend > middleware > JS Authmiddleware.js > ...
1  const admin = require("firebase-admin");
2
3  // Initialize Firebase Admin SDK once (safe to call multiple times – Admin SDK handles it)
4  if (!admin.apps.length) {
5      admin.initializeApp({
6          credential: admin.credential.cert({
7              projectId: process.env.FIREBASE_PROJECT_ID,
8              clientEmail: process.env.FIREBASE_CLIENT_EMAIL,
9              // The private key comes from .env as a single-line string with \n escapes
10             privateKey: process.env.FIREBASE_PRIVATE_KEY?.replace(/\\n/g, "\n"),
11         }),
12     });
13 }
14
15 async function verifyToken(req, res, next) {
16     const authHeader = req.headers.authorization;
17
18     // Expect: "Authorization: Bearer <token>"
19     if (!authHeader || !authHeader.startsWith("Bearer ")) {
20         return res.status(401).json({
21             error: "Non autorisé",
22             message: "Token d'authentification manquant."
23         });
24     }
25
26     const token = authHeader.split("Bearer ")[1];
27
28     try {
29         const decoded = await admin.auth().verifyIdToken(token);
30         // Attach the decoded user info to the request so controllers can use it
31         req.user = decoded;
32         next();
33     } catch (error) {
34         console.error("Erreur de vérification du token :", error.message);
35         return res.status(401).json({
36             error: "Non autorisé",
37             message: "Token invalide ou expiré."
38         });
39     }
40 }

```

3/ **translateRoutes.js** : Ce fichier, **translateRoutes.js**, visible dans la **image_922f62.png**, définit la structure des points d'accès (endpoints) de ton API pour les fonctionnalités de traduction et de chat.

Voici un résumé de son fonctionnement :

Architecture des Routes : **translateRoutes.js**

Ce script utilise le framework **Express** pour diriger les requêtes entrantes vers les services appropriés tout en appliquant une couche de sécurité systématique.

1. Configuration des Endpoints

Le fichier définit deux routes principales utilisant la méthode HTTP **POST** (lignes 8 et 9 de la **image_922f62.png**) :

- **/translate** : Dédiee aux requêtes de traduction de texte.
- **/chat** : Dédiee aux interactions avec l'assistant conversationnel.

2. Sécurisation par Middleware

La caractéristique majeure de ce fichier est l'intégration du middleware **verifyToken** (lignes 8-9) :

- **Contrôle d'accès** : Avant que la requête n'atteigne la logique métier (le contrôleur), elle passe par `verifyToken`.
- **Protection des ressources** : Si le jeton d'authentification est manquant ou invalide, le serveur renvoie immédiatement une erreur **401** (lignes 6-7).
- **Optimisation** : Le contrôleur ne s'exécute jamais pour les requêtes non autorisées, économisant ainsi les ressources du serveur et de l'IA.

3. Liaison avec la Logique Métier

Une fois l'utilisateur authentifié, les requêtes sont transmises au `translateController` :

- L'utilisation de `.bind(translateController)` garantit que les méthodes du contrôleur conservent leur contexte d'exécution correct lorsqu'elles traitent la requête.

```
backend > routes > JS translateRoutes.js > ...
1  const express = require("express");
2  const router = express.Router();
3  const translateController = require("../controllers/translateController");
4  const verifyToken = require("../middleware/authMiddleware");
5
6  // verifyToken runs before the controller on both routes
7  // If the token is missing or invalid, it returns 401 and the controller never runs
8  router.post("/translate", verifyToken, translateController.translate.bind(translateController));
9  router.post("/chat", verifyToken, translateController.chat.bind(translateController));
10
11 module.exports = router;
```

1. Le Constructeur : Préparation de l'IA

Le `constructor()` (lignes 4 à 26) prépare l'environnement dès le démarrage du service :

- **Sécurité des Clés (Lignes 5-8)** : Le code récupère la clé API via une variable d'environnement (`GEMINI_API_KEY`). Si la clé est absente, il bloque le démarrage avec une erreur claire, protégeant ainsi l'application contre des appels vides.
- **Connexion au SDK Google (Ligne 11)** : Il initialise l'objet `genAI` en utilisant la version `v1beta`, nécessaire pour accéder aux modèles les plus récents comme Gemini 2.x.
- **Stratégie de "Fallback" (Lignes 13-22)** : C'est une partie très intelligente du code. Tu as défini une liste de modèles candidats (`gemini-2.0-flash`, `gemini-1.5-pro`, etc.). Si le premier modèle de la liste ne fonctionne pas, le service pourra basculer sur le suivant.
- **Activation du Modèle (Lignes 24-25)** : Le service sélectionne le modèle par défaut et confirme l'initialisation réussie dans la console du serveur.

2. Gestion Intelligente des Erreurs

La méthode `_isModelError(error)` (lignes 28-35) est un utilitaire de diagnostic :

- **Analyse de Message** : Elle examine le message d'erreur renvoyé par les serveurs de Google.
- **Criblage (Lignes 31-33)** : Elle cherche des mots-clés spécifiques comme `404`, `not found` ou `not supported`.
- **Rôle** : Cette fonction permet au service de savoir si l'erreur vient d'un problème de modèle indisponible (ce qui déclenchera le passage au modèle suivant) plutôt que d'une erreur de réseau ou de code.

```

backend > services > JS aiService.js > ...
1  const { GoogleGenerativeAI } = require("@google/generative-ai");
2
3  class AIService {
4      constructor() {
5          const apiKey = process.env.GEMINI_API_KEY;
6          if (!apiKey) {
7              throw new Error('GEMINI_API_KEY manquante dans le fichier .env');
8          }
9
10         // Use v1beta - required for Gemini 2.x models
11         this.genAI = new GoogleGenerativeAI(apiKey, { apiVersion: 'v1beta' });
12
13         this.modelCandidates = [
14             process.env.GEMINI_MODEL,
15             'gemini-2.0-flash',
16             'gemini-2.0-flash-lite',
17             'gemini-1.5-flash',
18             'gemini-1.5-pro',
19         ].filter(Boolean);
20
21         this.currentModelIndex = 0;
22         this.modelName = this.modelCandidates[this.currentModelIndex];
23         // Fixed: getGenerativeModel only takes one argument (model config)
24         this.model = this.genAI.getGenerativeModel({ model: this.modelName });
25         console.log(` AI Service initialized with model: ${this.modelName}`);
26     }
27
28     isModelError(error) {
29         const message = error?.message?.toLowerCase() || '';
30         return (
31             message.includes('404') ||
32             message.includes('not found') ||
33             (message.includes('model') && message.includes('not supported'))
34         );
35     }
36

```

1. La Mécanique de Basculement : `_switchModel()`

Cette méthode (lignes 1 à 12) est le cœur de la résilience de ton service :

- **Vérification des limites (Lignes 2-4)** : Elle vérifie s'il reste des modèles disponibles dans la liste `modelCandidates`. Si tous les modèles ont été testés sans succès, elle renvoie `false`.
- **Incrémentation (Ligne 5)** : Elle passe à l'index suivant dans la liste.
- **Ré-initialisation (Lignes 6-8)** : Elle met à jour le nom du modèle actif et recrée une instance du modèle génératif via le SDK Google.
- **Journalisation (Ligne 9)** : Elle affiche un message dans la console du serveur pour tracer le changement de modèle (ex: "Changement de modèle Gemini vers : gemini-1.5-pro"), ce qui est précieux pour le débogage.

2. Clarification Linguistique : `_normalizeLanguage()`

Cette fonction (lignes 14 à 36) sert à "traduire" les codes de langue techniques en noms complets compréhensibles par l'IA :

- **Dictionnaire de mappage (Lignes 16-31)** : Elle contient une liste de correspondances, par exemple transformer `fr` en `French`, `ko` en `Korean` ou `ar` en `Arabic`.
- **Nettoyage des données (Ligne 34)** : Elle transforme l'entrée en chaîne de caractères, retire les espaces inutiles (`trim()`) et gère les cas de valeurs vides.

- **Sécurité (Ligne 35)** : Si le code langue n'est pas dans le dictionnaire, elle renvoie la valeur originale par défaut. Cela garantit que l'IA reçoit toujours une instruction de langue lisible, ce qui améliore la qualité de la traduction.

Note pour ton Audit (Disponibilité & Intégrité)

- **Disponibilité** : La fonction `_switchModel()` est une preuve directe de ta stratégie de haute disponibilité. Elle permet au backend de StudyShare de continuer à fonctionner même si un service spécifique de Google subit une panne locale.
- **Intégrité** : `_normalizeLanguage()` assure l'**Intégrité** des données envoyées à l'IA. En fournissant des noms de langues explicites (comme "Arabic") plutôt que des codes courts (comme "ar"), tu réduis considérablement le risque que l'IA interprète mal la demande de l'utilisateur.

```

_switchModel() {
  if (this.currentModelIndex + 1 >= this.modelCandidates.length) {
    return false;
  }
  this.currentModelIndex += 1;
  this.modelName = this.modelCandidates[this.currentModelIndex];
  // Fixed: same here - no second argument
  this.model = this.genAI.getGenerativeModel({ model: this.modelName });
  console.log(`Changement de modèle Gemini vers: ${this.modelName}`);
  return true;
}

_normalizeLanguage(language) {
  const languages = {
    ar: 'Arabic',
    de: 'German',
    en: 'English',
    es: 'Spanish',
    fr: 'French',
    it: 'Italian',
    ja: 'Japanese',
    ko: 'Korean',
    nl: 'Dutch',
    pt: 'Portuguese',
    ru: 'Russian',
    tr: 'Turkish',
    zh: 'Chinese',
  };

  const normalizedLanguage = String(language || '').trim();
  return languages[normalizedLanguage.toLowerCase()] || normalizedLanguage;
}

async _runPrompt(prompt) {
  let lastError = null;

```

1. Exécution avec Auto-Réparation : `_runPrompt()`

Cette méthode (lignes 1 à 18) est responsable de l'envoi effectif de la requête à Google Gemini. Elle utilise une boucle `while (true)` pour gérer les pannes intelligemment :

- **Tentative d'exécution (Lignes 3-7)** : Elle appelle `generateContent(prompt)` avec le modèle actuellement actif. Si tout va bien, elle renvoie le texte généré immédiatement.
- **Gestion des échecs (Lignes 8-15)** : En cas d'erreur, elle journalise l'échec dans la console.
- **Mécanisme de Basculement (Ligne 11)** : Si l'erreur provient du modèle (identifié par `_isModelError`) et qu'un autre modèle est disponible (via `_switchModel`), elle utilise l'instruction `continue` pour relancer immédiatement la même requête avec le nouveau modèle.
- **Sécurité** : Si aucun modèle ne fonctionne, elle finit par propager l'erreur (`throw lastError`) pour avertir le contrôleur.

2. Ingénierie de Requête : `translateText()`

Cette fonction (lignes 21 à 40) prépare le terrain pour obtenir la meilleure traduction possible :

- **Préparation des variables (Lignes 23-24)** : Elle normalise la langue cible et nettoie le contexte fourni par l'étudiant.
- **Contexte Facultatif (Lignes 25-27)** : Si l'étudiant a fourni un contexte (ex: "C'est un cours de réseau informatique"), le code génère une instruction spécifique pour guider l'IA vers le vocabulaire technique le plus naturel.
- **Le Prompt "Expert" (Lignes 28-39)** : C'est ici que tu définis le comportement de l'IA. Tu lui donnes un rôle précis ("You are an expert translator") et des consignes strictes pour éviter les explications inutiles et ne renvoyer que le texte traduit.

Note pour ton Audit (Disponibilité & Intégrité)

- **Disponibilité** : La boucle de relance dans `_runPrompt()` garantit que le service de traduction reste disponible même si l'un des modèles de Google est en maintenance ou surchargé.
- **Intégrité** : Les instructions système rigoureuses dans le `prompt` (ligne 28) assurent que l'IA ne dévie pas de sa mission. En lui ordonnant de ne renvoyer "que le texte traduit, sans explication", tu garantis que les données retournées au frontend sont propres et exploitables par l'interface utilisateur.

```

async _runPrompt(prompt) {
  while (true) {
    try {
      console.log('📢 Appel Gemini avec modèle ${this.modelName}');
      const result = await this.model.generateContent(prompt);
      const response = await result.response;
      return response.text();
    } catch (error) {
      console.error('❌ Erreur Gemini (${this.modelName}):', error.message);
      lastError = error;
      if (this._isModelError(error) && this._switchModel()) {
        continue;
      }
      break;
    }
  }
  throw lastError;
}

async translateText(text, language, context = '') {
  try {
    const targetLanguage = this._normalizeLanguage(language);
    const translationContext = String(context || '').trim();
    const contextInstructions = translationContext
      ? `Use this context to choose the most natural meaning, tone, register, and wording. Do not translate or repeat the context.`
      : '';
    const prompt = `You are an expert translator.
    Translate the text below into ${targetLanguage}.
    Return only the translated text, with no explanation.
    If context is provided, follow it carefully. If no context is provided and the meaning is ambiguous, use the most common/basic translation.`;
    const fullPrompt = `${contextInstructions}
    ${text}
    ${prompt}`;
    const result = await this._runPrompt(fullPrompt);
    return result;
  } catch (error) {
    return "[DEMO] Traduction en...";
  }
}

```

1. Sécurité de Repli (Fallback) pour la Traduction

Dans la continuité de la méthode `translateText` (lignes 1 à 10) :

- **Gestion des erreurs critiques** (Ligne 6) : Si l'IA ne répond pas après toutes les tentatives de basculement de modèle, le système attrape l'erreur (`catch`).
- **Mode Dégradé** (Ligne 8) : Au lieu de faire planter l'application, le code renvoie une chaîne de caractères spécifique "[DEMO] Traduction en...". Cela permet à l'utilisateur de comprendre que le service est en maintenance ou mal configuré sans interrompre son flux de travail.

2. L'Assistant Intelligent : `chatWithAI()`

Cette méthode (lignes 13 à 38) définit la personnalité et les limites de l'IA de **StudyShare** :

- **Définition du Périmètre (System Prompt)** : Le code définit précisément le rôle de l'IA comme "LinguaSense's translation assistant".
- **Restrictions de Sécurité (Lignes 16-24)** :
 - Le champ d'action est strictement limité à l'aide linguistique (grammaire, expressions, amélioration de style).
 - Le code ordonne explicitement à l'IA de refuser poliment toute demande non liée aux langues ou à la traduction. Cela garantit que l'étudiant reste concentré sur ses objectifs pédagogiques.
- **Adaptabilité Linguistique** (Lignes 26-28) : L'IA doit répondre dans la même langue que l'étudiant, assurant une expérience utilisateur fluide et naturelle.

3. Gestion des Erreurs et Exportation

- **Disponibilité du Chat (Lignes 32-35)** : En cas d'échec du service de chat, un message d'indisponibilité clair est renvoyé à l'utilisateur.
- **Exportation du Service (Ligne 41)** : Le fichier exporte une instance unique (`new AIService()`). Cela signifie que toute l'application partage la même configuration d'IA, optimisant ainsi l'utilisation de la mémoire du serveur.

```
async translateText(text, language, context = '') {
  const prompt = `You are an expert translator.

Text:
${text}`;
  return await this._runPrompt(prompt);
} catch (error) {
  console.error('Erreur lors de la traduction:', error.message);
  return `[DEMO] Traduction en ${language}: "${text}" - clé Gemini ou modèle non configuré correctement`;
}

async chatWithAI(message) {
  try {
    const prompt = `You are LinguaSense's translation assistant.
Your scope is strictly limited to translation-related help, including:
- translating text
- explaining words, expressions, idioms, grammar, tone, register, or cultural context
- improving or comparing translations
- helping choose a source/target language or wording style

If the user's message is not related to translation, language, writing, grammar, or localization, politely refuse and briefly say you
Answer in the same language as the user's message. If the user asks for a specific response language, use that language.
Keep the answer concise and useful.

User message:
${message}`;
    return await this._runPrompt(prompt);
  } catch (error) {
    console.error('Erreur lors du chat:', error.message);
    return 'The translation assistant is currently unavailable. Please check the Gemini API key or model configuration.';
  }
}

module.exports = new AIService();
```

4/ **.ENV** : Le fichier `.env` présenté dans la **Capture d'écran 2026-05-14 153230.png** est le coffre-fort de ton application. Il contient toutes les variables sensibles et les configurations qui ne doivent pas être écrites directement dans le code pour des raisons de sécurité.

Catégorie	Variables clés	Rôle et Utilité
Firestore (Backend)	FIREBASE_PROJECT_ID, PRIVATE_KEY	Permet au serveur de s'authentifier de manière sécurisée auprès de Google Cloud pour gérer la base de données Firestore.
Intelligence Artificielle	GEMINI_API_KEY, GEMINI_MODEL	Contient la clé secrète pour appeler l'IA Gemini et définit quel modèle de langage (ex: gemini-flash-latest) doit être utilisé.
Paramètres Système	NODE_ENV, PORT	Définit l'environnement de travail (ici development) et le port réseau (ici 3000) sur lequel le serveur écoute les requêtes.

```
backend > .env
1 # Firebase Service Account Configuration
2 FIREBASE_PROJECT_ID=linguasense-726ba
3 FIREBASE_CLIENT_EMAIL=firebase-adminsdk-fbsvc@linguasense-726ba.iam.gserviceaccount.com
4 FIREBASE_PRIVATE_KEY="-----BEGIN PRIVATE KEY-----\nMIIEvAIBADANBgkqhkiG9w0BAQEFAASCCKYwgSIAgEAAoIBAQDvGC7mBQxQ01p5\n5
6 # AI API
7 GEMINI_API_KEY=AIzaSyD0o-eE6VoHGgvfvjylohJPb5F83rmQ8kg
8 GEMINI_API_URL=https://generativelanguage.googleapis.com/v1beta/models/gemini-flash-latest:generateContent
9 GEMINI_MODEL=gemini-flash-latest
10
11 # App settings
12 NODE_ENV=development
13 PORT=3000
```

3.5 Pattern MVC du backend

Le backend applique une architecture MVC claire qui sépare les responsabilités :

Couche	Fichier	Responsabilité
Routes	translateRoutes.js	Définit POST /api/translate et POST /api/chat, applique le middleware auth
Middleware	authMiddleware.js	Vérifie le token Firebase (Bearer), retourne 401 si invalide ou absent
Contrôleur	translateController.js	Valide les paramètres, appelle aiService, formate la réponse JSON
Service	aiService.js	Initialise Gemini, construit les prompts, gère le fallback entre modèles

4. Fonctionnalités Réalisées

4.1 Authentification utilisateur (Firebase Auth)

L'ensemble du système d'authentification est géré via Firebase Authentication côté frontend, et Firebase Admin SDK côté backend :

- Inscription avec email, mot de passe et nom (`createUserWithEmailAndPassword` + `updateProfile`)
- Connexion (`signInWithEmailAndPassword`) avec redirection vers l'interface principale
- Déconnexion (`signOut`) avec mise à jour dynamique de la navbar
- Redirection automatique : utilisateur non connecté → `login.html`, déjà connecté sur `login` → `index.html`
- Messages d'erreur en français pour chaque code Firebase (email déjà utilisé, mauvais mot de passe, etc.)
- Vérification du token côté backend via `admin.auth().verifyIdToken()` avant chaque requête protégée

4.2 Traduction intelligente via Gemini

La fonctionnalité centrale de l'application repose sur un appel sécurisé à l'API Gemini :

- L'utilisateur saisit un texte, sélectionne les langues source et cible, et un contexte optionnel
- Le frontend récupère un token Firebase frais (`currentUser.getIdToken()`) et l'envoie dans le header `Authorization: Bearer`
- Le backend vérifie le token, contrôle la limite gratuite, construit le prompt et appelle Gemini
- La traduction est retournée et affichée ; l'entrée est sauvegardée dans Firestore automatiquement

Prompt envoyé à Gemini : "Tu es un traducteur expert. Traduis ce texte en [langue] : [texte]"

4.3 Système de fallback Gemini

Le service `aiService.js` implémente un mécanisme de robustesse : si un modèle Gemini renvoie une erreur 404 ou "not supported", le service passe automatiquement au modèle suivant.

Priorité	Modèle	Condition
1	Variable GEMINI_MODEL (.env)	Si définie dans les variables d'environnement
2	gemini-2.0-flash	Modèle principal par défaut
3	gemini-2.0-flash-lite	Fallback si le précédent est indisponible
4	gemini-1.5-flash	Fallback secondaire
5	gemini-1.5-pro	Dernier recours

4.4 Historique des traductions (Firestore)

Chaque traduction réussie est automatiquement sauvegardée dans Firestore dans une sous-collection privée par utilisateur :

Collection : users/{uid}/translations

Champ	Type	Description
originalText	String	Texte saisi par l'utilisateur
translatedText	String	Traduction retournée par Gemini
sourceLanguage	String	Code de la langue source (fr, en, es...)
targetLanguage	String	Code de la langue cible
timestamp	Timestamp	Date et heure du serveur (serverTimestamp())

La page history.html affiche les traductions triées par date décroissante (orderBy timestamp desc). Chaque entrée peut être supprimée individuellement via deleteDoc().

4.5 Modèle d'abonnement (freemium)

Un système freemium simple est implémenté pour simuler un modèle économique réel :

- Chaque utilisateur dispose d'un champ isSubscribed dans Firestore
- Le backend vérifie le nombre de traductions déjà effectuées
- Au-delà de 15 traductions gratuites, l'utilisateur est redirigé vers upgrade.html
- La page upgrade.html présente les offres premium disponibles
- Note : le paiement réel n'est pas intégré dans la V1

4.6 Assistant IA

Un second endpoint POST /api/chat permet à l'utilisateur de poser des questions à un assistant IA contextuel lié au projet LinguaSense. Le prompt envoyé à Gemini intègre le message utilisateur dans un contexte prédéfini.

5. Flux Technique — Parcours d'une Traduction

Étape	Acteur	Action
1	Utilisateur	Saisit le texte, choisit les langues et le contexte, clique "Traduire"
2	Frontend (translate.js)	Vérifie que l'utilisateur est connecté (onAuthStateChanged)
3	Frontend	Récupère un token Firebase frais : currentUser.getIdToken()
4	Frontend	Envoie POST /api/translate avec Authorization: Bearer <token> et body JSON
5	Backend (middleware)	Vérifie le token via admin.auth().verifyIdToken() — renvoie 401 si invalide
6	Backend (contrôleur)	Valide les paramètres requis (text, language)
7	Backend (contrôleur)	Contrôle la limite gratuite et le statut abonnement dans Firestore
8	Backend (aiService)	Construit le prompt et appelle l'API Gemini
9	Gemini API	Génère et retourne la traduction
10	Backend	Renvoie { success, originalText, translatedText, language }
11	Frontend	Affiche la traduction dans l'interface
12	Frontend (Firestore)	Sauvegarde l'entrée dans users/{uid}/translations via addDoc()

5.1 Sécurité du flux

- Les clés API (Gemini, Firebase Admin) sont stockées dans .env, jamais exposées côté client
- Le token Firebase a une durée de vie limitée ; il est renouvelé automatiquement par le SDK
- Toutes les routes /api/ sont protégées par le middleware d'authentification
- L'historique est stocké dans une sous-collection privée par UID : aucun accès croisé possible

6. Pages de l'Application

Page	Fichier HTML	Description
Traducteur	index.html	Interface principale : saisie, langues, contexte, traduction, assistant IA
Connexion	login.html	Formulaire de connexion avec messages d'erreur en français
Inscription	register.html	Formulaire d'inscription (nom, email, mot de passe)
Historique	history.html	Liste des traductions passées triées par date, avec suppression individuelle
Premium	upgrade.html	Présentation des offres d'abonnement et redirection si limite atteinte

6.1 États d'interface gérés

Situation	Comportement
Chargement traduction	Bouton désactivé + texte "Traduction en cours..." pendant l'appel API
Texte manquant	Message d'erreur rouge : "Veuillez saisir un texte à traduire."
Langue cible manquante	Message d'erreur rouge : "Veuillez choisir une langue cible."
Utilisateur non connecté	Redirection automatique vers login.html
Erreur serveur (401)	Redirection vers login.html (token expiré ou invalide)
Erreur API (autre)	Affichage du message d'erreur retourné par le backend
Historique vide	"Aucune traduction sauvegardée pour l'instant."
Chargement historique	Message "Chargement..." pendant la requête Firestore
Limite gratuite atteinte	Redirection vers upgrade.html

7. Rôle de l'IA dans le Projet

7.1 IA comme outil de cadrage (Copilot 365)

Avant d'écrire la moindre ligne de code, l'IA a servi à structurer le projet selon la méthode enseignée en cours. Cette approche évite de coder un mauvais produit rapidement.

- **Génération de 12 questions de découverte classées par thème (objectifs, utilisateurs, fonctionnalités, langues, contraintes, performance, périmètre)**
- Analyse des réponses client et reformulation en brief amélioré
- Séparation V1 / Plus tard / Hors périmètre avec justification de chaque choix
- Définition du parcours utilisateur en 12 étapes
- Architecture technique recommandée et validée
- Identification des risques et critères d'acceptation

7.2 IA comme moteur de traduction (Gemini)

L'API Gemini est le cœur technique de l'application. Elle est appelée depuis le backend via le package officiel `@google/generative-ai` :

- Traduction contextuelle : le prompt intègre le texte, la langue cible et (à terme) le contexte saisi
- Réponses de l'assistant IA : endpoint `/api/chat` avec prompt contextualisé autour de LinguaSense
- Robustesse : fallback automatique entre 4 modèles Gemini en cas d'indisponibilité
- Flexibilité : le modèle utilisé est configurable via la variable `GEMINI_MODEL` dans `.env`

7.3 IA pour le développement

L'IA a également assisté la phase de développement :

- Génération et débogage du code backend (Express, middleware, services)
- Aide à la structuration du pattern MVC
- Rédaction de la documentation et du README
- Résolution d'erreurs (CORS, tokens Firebase, modèles Gemini indisponibles)

8. Bilan du Projet

8.1 Fonctionnalités livrées

Fonctionnalité	Statut	Remarque
Inscription / Connexion Firebase	Livré	
Déconnexion + mise à jour navbar	Livré	
Traduction via Gemini	Livré	
13 langues disponibles	Livré	
Champ contexte dans l'interface	Livré	Intégration dans le prompt à finaliser
Assistant IA (/api/chat)	Livré	
Historique (lecture + suppression)	Livré	
Sauvegarde auto dans Firestore	Livré	
Limite gratuite (15 traductions)	Livré	
Page premium upgrade.html	Livré	Sans paiement réel
Sécurisation des routes (tokens)	Livré	
Fallback automatique entre modèles	Livré	
Édition intégrée de la traduction	Partiel	Reprise manuelle possible, UI à finaliser
Paiement réel	Hors V1	Évolution future

8.2 Ce que ce projet m'a appris

- Cadrer un projet avec l'IA avant de coder : questions, périmètre, artefacts
- Intégrer une API LLM externe (Gemini) via un backend Node.js sécurisé
- Structurer un backend en architecture MVC (Routes / Middleware / Controller / Service)
- Gérer l'authentification Firebase end-to-end (client + serveur)
- Persister et lire des données en temps réel avec Firestore
- Sécuriser les clés API avec des variables d'environnement
- Construire un système freemium simple avec contrôle côté serveur

8.3 Évolutions futures planifiées

Évolution	Priorité
Intégration complète du champ contexte dans le prompt Gemini	Haute
Édition intégrée de la traduction dans l'interface	Haute
Import / export de fichiers (PDF, Word)	Moyenne
Paiement réel pour les abonnements	Moyenne
Personnalisation avancée (ton, domaine métier, profil)	Moyenne

Apprentissage basé sur les corrections utilisateurs	Basse
API publique pour intégrations externes	Basse
Application mobile (PWA ou React Native)	Basse
Nouvelles langues et variantes régionales	Basse

9. Conclusion

Le projet LinguaSense démontre qu'il est possible de livrer un MVP complet, sécurisé et fonctionnel en une semaine, à condition de bien cadrer le besoin avant de coder. L'utilisation de l'IA en phase de découverte a permis d'éviter les dérives classiques (trop de fonctionnalités, périmètre flou, priorités mal définies) et de produire une V1 cohérente avec les attentes du client.

Techniquement, le projet combine les briques essentielles d'une application web moderne : un backend Express structuré en MVC, une authentification sécurisée via Firebase (Auth + Admin SDK), un stockage NoSQL avec Firestore, un modèle LLM (Gemini) comme moteur intelligent, et un système freemium fonctionnel.

Ce qui distingue ce projet d'un simple exercice technique, c'est la double utilisation de l'IA : comme outil de réflexion produit (Copilot 365 pour le cadrage), et comme composant technique central (Gemini pour la traduction). Cette approche — cadrer avec l'IA, construire avec l'IA — est précisément ce que ce module nous a appris à maîtriser.

Résumé : une app web de traduction intelligente avec Gemini, Firebase Auth, Firestore, historique, freemium et assistant IA — réalisée en 1 semaine, cadrée avec l'IA, construite avec l'IA.